

PyTorch中自动微分（automatic differentiation）



蔚蓝小栈

武汉大学 摄影测量与遥感博士

+ 关注他

4 人赞同了该文章

Pytorch中的每一个张量都有一个 `requires_grad` 属性，“传染性”意味着如果一个张量是通过其他张量的操作创建的，并且至少有一个参与操作的张量的 `requires_grad` 属性被设置为 `True`，那么结果张量的 `requires_grad` 属性也会自动被设置为 `True`。

例子说明

创建张量:

- 假设有一个张量 `a`，并且我们设置它的 `requires_grad` 为 `True`
`a = torch.randn((3, 3), requires_grad=True)`

通过操作创建新张量:

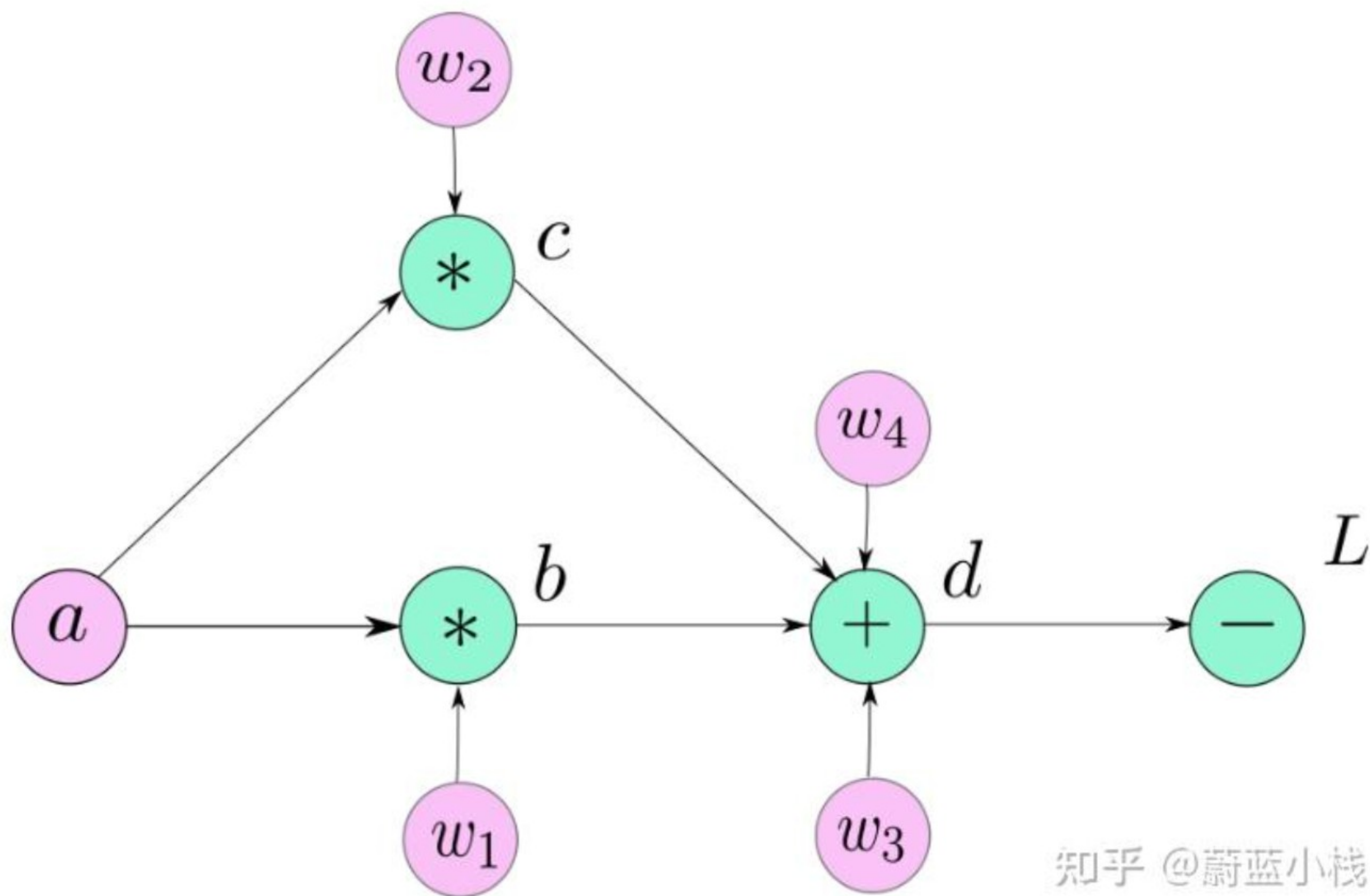
- 如果我们对 `a` 进行某种操作来创建新张量 `b`
`b = a * 2` # 这里, `b` 是通过对 `a` 的操作创建的
- 由于 `a` 的 `requires_grad` 为 `True`，因此 `b` 的 `requires_grad` 也会自动被设置为 `True`。

无需手动跟踪:

- 这意味着程序员在进行计算时，不必手动跟踪每个张量的 `requires_grad` 状态。只需确保输入张量的 `requires_grad` 属性正确设置，输出张量会自动继承这一状态。

现在假定我们有一个神经网络，在pytorch中自动微分要如何计算得到梯度的呢？

如果我们把这个计算图中的`a,w1,w2,w3,w4`看成是用户给定的输入，这些输入节点之外的，其它图中的节点叫做操作符，而对应的边叫做输出的话，我们可以得到上面的计算公式，以及对所有可以学习参数的梯度。如果我们反向来看，我们要对任意的变量`a`求梯度的时候，实际上是将所有路径上的梯度的乘积结果求和



一个简单的神经网络

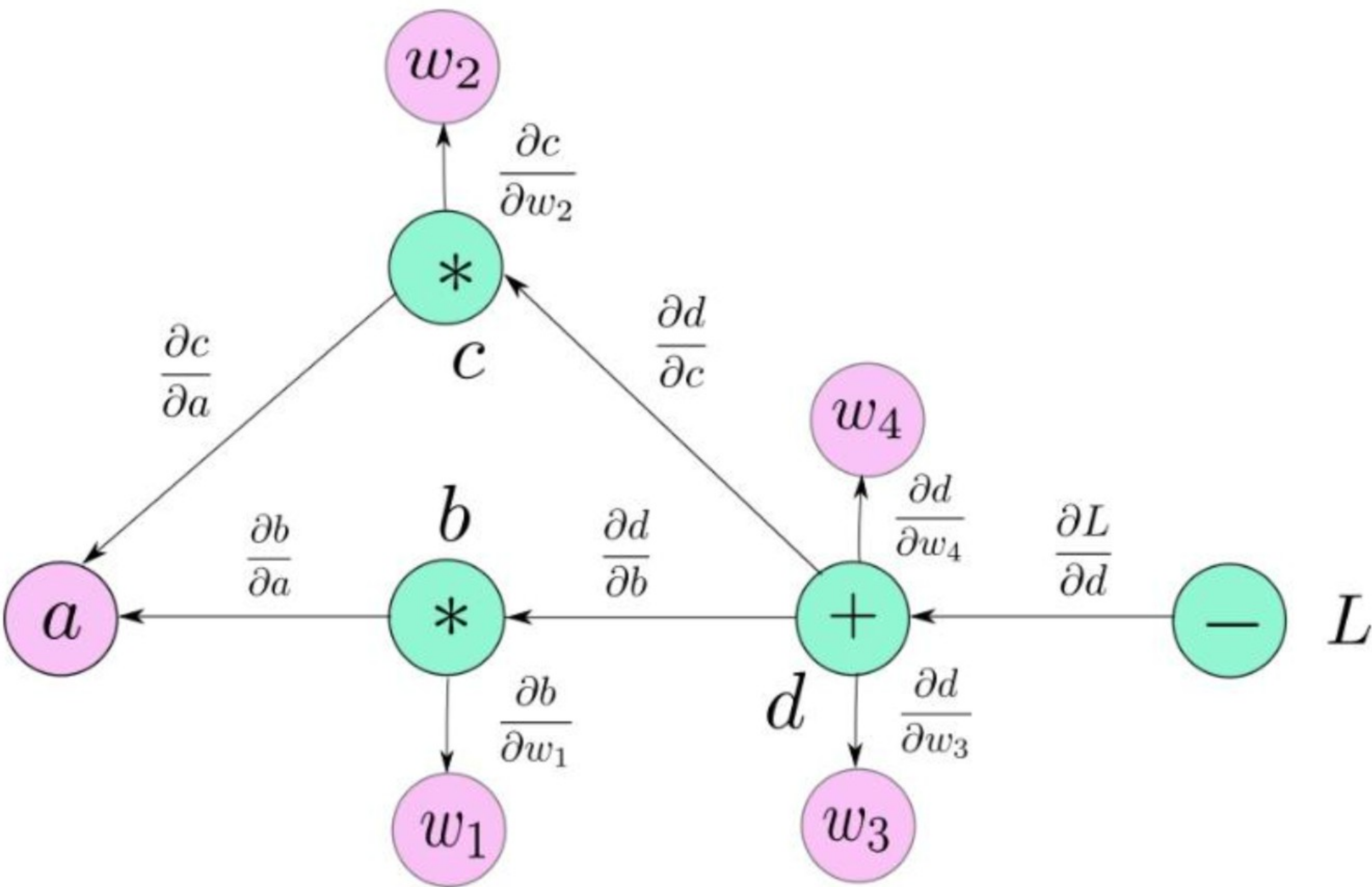
图 1 一个简单的神经网络

$$\begin{aligned} b &= w_1 * a \\ c &= w_2 * a \\ d &= w_3 * b + w_4 * c \\ L &= 10 - d \end{aligned}$$

Let us compute the gradients for each of the learnable parameters w .

$$\begin{aligned} \frac{\partial L}{\partial w_4} &= \frac{\partial L}{\partial d} * \frac{\partial d}{\partial w_4} \\ \frac{\partial L}{\partial w_3} &= \frac{\partial L}{\partial d} * \frac{\partial d}{\partial w_3} \\ \frac{\partial L}{\partial w_2} &= \frac{\partial L}{\partial d} * \frac{\partial d}{\partial c} * \frac{\partial c}{\partial w_2} \\ \frac{\partial L}{\partial w_1} &= \frac{\partial L}{\partial d} * \frac{\partial d}{\partial b} * \frac{\partial b}{\partial w_1} \end{aligned}$$

如果我们把这个计算图中的 a, w_1, w_2, w_3, w_4 看成是用户给定的输入，这些输入节点之外的，其它图中的节点叫做操作符，而对应的边叫做输出的话，我们可以得到上面的计算公式，以及对所有可以学习参数的梯度。

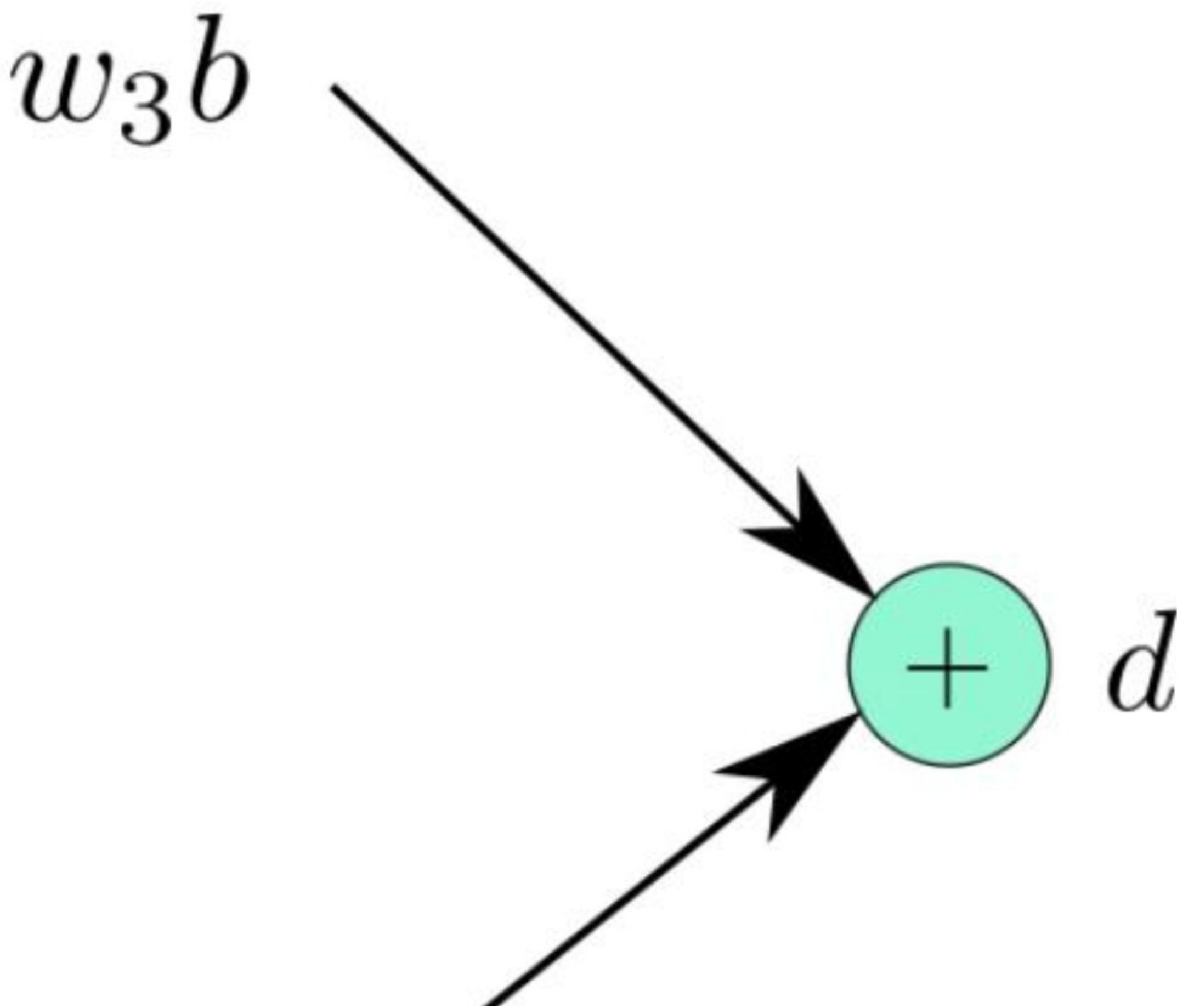


Backpropagation in a Computational Graph

知乎 @蔚蓝小栈

如果我们反向来看，我们要对任意的变量a求梯度的时候，实际上是将所有路径上的梯度的乘积结果求和

$$d = f(w_3b, w_4c)$$



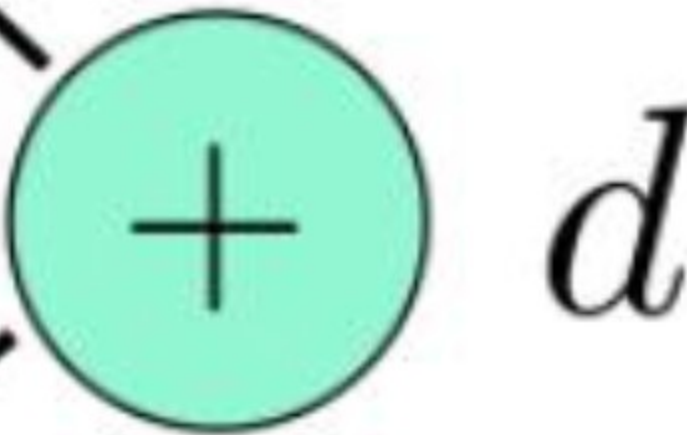
w_4c 

d is output of function $f(x,y) = x + y$ 知乎 @蔚蓝小栈

由于d是w3b和w4c之和

w_3b

$\frac{\partial d}{\partial w_3b}$



w_4c

$\frac{\partial d}{\partial w_4c}$

Local Gradients 知乎 @蔚蓝小栈

假设边上面的公式 $\partial d/(\partial w_3b)$ 和 $\partial d/(\partial w_4c)$ 是d相对于自己局部输入的梯度

反向传播过程的总结

1. 梯度存储:
- 在反向传播中，前面一层的梯度（例如，损失函数相对于当前层输出的梯度）会存储在当前层的输出变量的`grad`属性中。这个梯度通常表示为 $\frac{\partial L}{\partial d}$ ，其中 L 是损失函数， d 是当前层的输出。
2. 使用梯度:
- 每次反向传播时，当前层的`backward`函数会使用这个存储在`grad`属性中的梯度。这个梯度是从后续层传递回来的，用于计算当前层的输入梯度。
3. 计算局部梯度:
- 当前层的`backward`函数会计算当前函数的局部梯度（例如， $\frac{\partial d}{\partial w_3b}$ 和 $\frac{\partial d}{\partial w_4c}$ ），这些局部梯度反映了当前层输出相对于其输入的变化。
4. 梯度相乘:
- 然后，将前面一层传递来的梯度与当前层计算出的局部梯度相乘，得到当前层输入的梯度。例如：
 - $\frac{\partial L}{\partial w_3b} = \frac{\partial L}{\partial d} \cdot \frac{\partial d}{\partial w_3b}$
 - $\frac{\partial L}{\partial w_4c} = \frac{\partial L}{\partial d} \cdot \frac{\partial d}{\partial w_4c}$
5. 梯度传递:
- 计算得到的输入梯度会继续传递到当前函数的输入张量中。这是通过调用输入张量的`grad\_fn`的`backward`方法来实现的，反向传播的过程会在整个计算图中递归进行。

反向传播的梯度计算和传递过程

也就是说前面一层的梯度会存储在当前层的输出变量d的grad属性当中，每次反向传播的时候，都会使用这个梯度，然后将当前函数的局部梯度计算出来之后，和这个梯度相乘之后，将对应的结果也继续传递到当前函数的输入张量中（在这个例子中是b和c），也就是当前层的前一个层的输出变量中。

使用 PyTorch 进行张量操作的时候，只能对标量（scalar）值的张量调用 backward() 方法，而不能对向量（vector）值的张量调用backward()。以下是样本代码和相关概念的详细解释：


```
import torch

# 创建一个 3x3 的张量 a, 设置 requires_grad 为 True 以便计算梯度
a = torch.randn((3,3), requires_grad=True)

# 创建四个 3x3 的权重张量, 设置 requires_grad 为 True
w1 = torch.randn((3,3), requires_grad=True)
w2 = torch.randn((3,3), requires_grad=True)
w3 = torch.randn((3,3), requires_grad=True)
w4 = torch.randn((3,3), requires_grad=True)

# 计算 b 和 c, 分别是 w1 和 w2 与 a 的乘积
b = w1 * a
c = w2 * a

# 计算 d, d 是 b 和 c 的加权和
d = w3 * b + w4 * c

# 计算损失 L, L 是 10 减去 d
L = (10 - d)

# 尝试对 L 调用 backward()
L.backward() # 这里会抛出错误
```

关键点

张量的创建:

a、w1、w2、w3 和 w4 都是 3x3 的张量，并且都设置了 requires_grad=True，这意味着它们将参与梯度计算。

计算过程:

b 和 c 是通过将 a 与权重张量 w1 和 w2 相乘得到的。

d 是通过将 b 和 c 加权相加得到的。

最后，损失 L 是通过从 10 中减去 d 得到的。

只能对标量调用 backward():

在 PyTorch 中，backward() 方法只能在标量（单个数值）张量上调用。如果尝试在向量（例如，3x3 的张量）上调用 backward()，将会抛出错误。

在这个例子中，L 是一个 3x3 的张量，因此调用 L.backward() 会导致错误。

如何解决

要解决这个问题，您可以将 `L` 修改为一个标量。例如，可以通过计算 `L` 的平均值或总和来实现：

```
L = (10 - d).mean() # 计算 L 的平均值
L.backward() # 现在可以成功调用
```

综上：所谓的自动微分，是指pytorch根据计算图，以及图与图中的节点之间连接关系，将前一层的梯度与当前层的局部梯度的积相乘，然后根据路径关系，将所有路径中的梯度乘积相加的操作，直到得到每一个非操作符变量的梯度的过程。